



ADR-012: Deployment — containers; Kubernetes as primary target; Railway for pilots

TASCO Insurance × VETC Growth Platform — Architecture Decision Record

iorta TechNXT

Version 1.0 · 07 October 2026

Contents

Context and problem statement

Decision drivers

Considered options

Decision outcome

Pros and cons of the options

- **Status:** Accepted
- **Date:** 2026-10-07
- **Related:** [Deployment & infrastructure architecture](#)

Context and problem statement

TASCO and VETC need production hosting that meets insurer InfoSec requirements: network segmentation, secrets management, HA, audit, and data residency in Vietnam (**confirm with TASCO legal:** Cybersecurity Law 2018 and Decree 53/2022/ND-CP data localisation; Decree 13/2023 and the PDP Law on cross-border transfer). For pilots and stakeholder demos, a low-friction PaaS is valuable. The same artefact must run in both places.

Decision drivers

- Build once and run anywhere: dev laptop, CI, PaaS, Kubernetes.
- Non-root, read-only, minimal images.
- Horizontal scaling and zero-downtime deploys.
- Portability between Vietnamese cloud providers (e.g. Viettel IDC, FPT Cloud, VNPT) and hyperscalers with a Vietnam presence.

Considered options

1. **OCI container (multi-stage, `node:22-alpine`, non-root) → Kubernetes (primary) with Kustomize manifests; Railway for pilots; docker-compose for local.**
2. VMs with systemd + Ansible.
3. Serverless functions.

Decision outcome

Chosen option: **1.**

Asset	Purpose
Dockerfile	Multi-stage. Production dependencies only (<code>npm ci --omit=dev</code>), <code>node:22-alpine</code> , runs as a non-root user, <code>NODE_ENV=production</code> , exposes 3000, <code>CMD node src/server.js</code>
docker-compose.yml	Local app + Postgres 16
deploy/k8s/	Namespace, Deployment (probes <code>/health/live</code> and <code>/health/ready</code> , requests/limits, <code>securityContext</code> with <code>readOnlyRootFilesystem</code> , <code>runAsNonRoot</code> , dropped capabilities), Service, Ingress (TLS), HPA, PDB, NetworkPolicy, CronJobs (journeys, reconcile, retention, relay), migration Job, ExternalSecret / secret template, ConfigMap
railway.json	Pilot deploy (Dockerfile builder, healthcheck <code>/health/ready</code>)
.github/workflows/ci.yml	Lint → tests with coverage gate → Postgres tests → CodeQL → npm audit and dependency review → gitleaks → image build → Trivy → ZAP baseline → SBOM

Application properties that make this work:

- 12-factor config with `*_FILE` secrets (`src/shared/config.js`);
- JSON logs on stdout;
- graceful `SIGTERM` handling: readiness drops first, then the server closes, with a forced exit after 25 s (`src/server.js`);
- stateless replicas (the outbox relay is safe on N replicas);
- jobs are the same image with a different command (`src/jobs/cli.js`).

Consequences

- Good: one immutable artefact, signed and scanned, is promoted from SIT to prod.
- Good: Kubernetes gives HPA, PDB, NetworkPolicy and secret-store integration, which InfoSec expects.
- Bad: Kubernetes operational overhead. A managed control plane is recommended.
- Bad / constraints:
 - Railway has no NetworkPolicy and offers limited control of data residency. It is **pilot only, with synthetic or consented data**. Production personal data stays in Vietnam-hosted Kubernetes and Postgres (**confirm with TASCO legal**).
 - The in-process rate limiter and token revocation are per replica, so multi-replica deployments need gateway rate limiting and a shared store ([security gaps](#)).
 - `MIGRATE_ON_START` defaults to running migrations at boot. In Kubernetes, set `MIGRATE_ON_START=false` and use the migration Job, because concurrent replicas could otherwise race (`migrate()` takes no lock).
 - The read-only root filesystem works because the app writes nothing to disk. The `openapi` job writes to `docs/api/` and is run in CI, not in the cluster.

Pros and cons of the options

VMs. Familiar to some infrastructure teams. However, they are slower to scale, give weaker immutability, and need more patching.

Serverless. Scales to zero. However, cold starts, Postgres connection storms, the long-running relay loop and data-residency limits make it a poor fit.